

Arduino and ESP32

Code Review Sample

Technical review, documentation improvement and compilation-readiness assessment

Project type	Personal demonstration
Scope	Review of example ESP32 temperature-monitoring code and setup documentation
Deliverable	Structured findings, revised code example and practical recommendations

Project objective

Assess a simple ESP32 analogue temperature-monitoring example for code clarity, compilation readiness, configuration requirements and documentation quality. The review focuses on whether a new user could understand the setup and reproduce the project.

Review activities

- Reviewed board configuration, pin definitions and serial settings.
- Checked the code structure for readability and maintainability.
- Identified missing validation and troubleshooting guidance.
- Prepared a revised example using clearer constants and basic input checking.
- Converted the findings into a concise technical report suitable for a client.

Important limitation

This sample demonstrates code review and technical documentation only. Physical operation was not verified on a connected microcontroller or sensor.

Review findings

The following issues were identified in a typical beginner ESP32 temperature-sensor example.

Review area	Finding	Recommendation
Board configuration	The required ESP32 board variant was not stated.	Specify the board package and selected board before compilation.
Pin assignment	The analogue input pin was hard-coded inside the program.	Use a named constant so the connection is easy to update and review.
Serial settings	The baud rate appeared only in code.	State the baud rate clearly in the setup instructions.
Input validation	No check was included for readings outside the expected ADC range.	Add a basic validity check and a clear warning message.
Documentation	No wiring table or upload troubleshooting was provided.	Add pin connections, board/port selection and common upload fixes.

Example code reviewed

```
const int sensorPin = 4;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int value = analogRead(sensorPin);
  float voltage = value * (3.3 / 4095.0);
  float temperatureC = voltage * 100.0;
  Serial.println(temperatureC);
  delay(1000);
}
```

Main observations

- The code is concise and suitable for a basic demonstration.
- The conversion assumes a 3.3 V ADC reference and 12-bit range.
- The sensor type and wiring are not documented inside the example.
- There is no detection of disconnected, saturated or implausible readings.

Revised example and delivery approach

A clearer version improves maintainability while keeping the project simple.

```
constexpr int SENSOR_PIN = 4;
constexpr unsigned long SERIAL_BAUD = 115200;
constexpr float ADC_REFERENCE_V = 3.3f;
constexpr int ADC_MAX = 4095;

void setup() {
  Serial.begin(SERIAL_BAUD);
  analogReadResolution(12);
}

void loop() {
  const int raw = analogRead(SENSOR_PIN);

  if (raw <= 0 || raw >= ADC_MAX) {
    Serial.println("Warning: check sensor wiring or input range.");
    delay(1000);
    return;
  }

  const float voltage = raw * (ADC_REFERENCE_V / ADC_MAX);
  const float temperatureC = voltage * 100.0f;

  Serial.print("Temperature: ");
  Serial.print(temperatureC, 1);
  Serial.println(" C");
  delay(1000);
}
```

Changes made

Improvement	Reason
Named constants	Makes the pin, baud rate and ADC assumptions visible and easy to change.
ADC configuration	States the intended resolution directly in the setup process.
Basic validation	Flags obviously invalid or saturated input readings.
Clear serial output	Adds labels and units so results are easier to interpret.

Example client deliverables

- Compilation and configuration checklist
- Concise code-review findings
- Reproduction steps for identified errors
- README or setup-guide improvements
- Screenshots or logs supplied by the client
- Editable Word report and final PDF

Skills demonstrated

ESP32 | Arduino | C++ | Embedded systems | Code review | Technical documentation | Troubleshooting | Report writing